

Apunte 18 — Git y GitHub: puntos de guardado y respaldo en la nube

Proyecto Froth · aprender Machine Learning construyendo

1 Qué es Git

Concepto: Git = máquina de puntos de guardado

Como en un videojuego: cada vez que llegas a un estado que vale la pena conservar, haces un **commit** (“guardar partida”): Git fotografía TODOS los archivos del proyecto en ese instante y guarda la foto para siempre, con fecha, autor y un mensaje. Si mañana rompes algo, puedes volver a cualquier foto anterior. El historial vive en la carpeta oculta `.git`. Lo usa todo programador del planeta, a diario.

Vocabulario mínimo:

- **repo(sitorio)**: el proyecto vigilado por Git.
- `git add .`: poner archivos “en la bandeja” para la próxima foto.
- `git commit -m "mensaje"`: disparar la foto.
- `git status`: ver qué hay en la bandeja / qué cambió.
- **rama (branch)**: una línea de historia. La principal se llama `main`.
- **remote / origin**: el apodo de la copia en la nube.
- `git push`: empujar tus fotos locales a la nube.

Concepto: `.gitignore` = lo que JAMÁS entra en las fotos

Un archivo de texto con la lista de lo prohibido: secretos (`.openalex_key`), la burbuja (`.venv/`), datos regenerables (`2_Datos/`), herramientas descargadas (`tools/`). Así el repo lleva el *código y el conocimiento*, no los pesos muertos ni las llaves. Verificamos con `git ls-files` que ningún secreto entró: cero.

2 Qué es GitHub (y qué NO es)

GitHub es la nube donde respaldas los repos de Git — y además tu **portafolio público**: cualquiera puede ver tu código, clonarlo (`git clone`) y correrlo en su PC. GitHub *guarda* el código; no lo *ejecuta* por el visitante.

Tu repo: `github.com/kmortizva-data/froth` — de momento **privado** (GitHub muestra 404 a los extraños para no revelar ni que existe). Cuando esté pulido: Settings → Change visibility → Public, y nace el portafolio.

3 Lo que pasó en tu primera vez (incluye los tropiezos, que enseñan)

1. `git init + git add . + git commit` → primera foto: 65 archivos, cero secretos.

2. **Tropezco 1:** el push falló con “repository does not seem to exist” — el remoto apuntaba a una URL de ejemplo cuya página nunca se creó en GitHub. Lección: *el repo en la nube hay que crearlo antes de empujar* (o dejar que GitHub Desktop lo cree con su botón **Publish repository**, que hace ambas cosas).
3. **Tropezco 2:** `git remote remove origin` respondió “No such remote” — significaba “ya estaba borrado”, no un error real. Lección: lee el mensaje; a veces el “error” es solo un aviso de que no había nada que hacer.
4. `git branch -M main`: renombrar `master` → `main` (el estándar moderno). El silencio de la terminal = éxito.
5. **Publish repository** en GitHub Desktop → creó el repo en la nube y empujó. Verificado: `main...origin/main` sin diferencias = sincronizado.

4 Tu rutina desde ahora (la única que necesitas memorizar)

Al final de cada sesión de trabajo con cambios que valgan:

```
git add . → git status (revisar) → git commit -m "qué hice" → git push
```

(O en GitHub Desktop: revisar cambios → escribir resumen → Commit → Push. Es lo mismo con botones.)

En una frase

Git guarda fotos completas del proyecto (commits) y GitHub las respalda en la nube y las exhibe como portafolio; el `.gitignore` protege secretos y pesos muertos. Tu rutina: `add` → `status` → `commit` → `push`. Primera foto: 65 archivos, cero secretos, ya en la nube.